

DANG TRAN NAM

Software Engineer · Ho Chi Minh City, Vietnam

nam.dt161@gmail.com · [0981405314](tel:0981405314) · [GitHub](#) · [LinkedIn](#)

Software Engineer specializing in backend systems, high-concurrency database architectures, and production full-stack SaaS. Experienced in designing resilient distributed agents, low-latency transaction engines, enterprise .NET backends, and Telegram Mini Apps.

Experience

An Binh Hospital
Software Engineer Intern – IT Asset Management (ITAM): Distributed Telemetry System - Distributed Telemetry Agent: Designed and developed a cross-platform Go agent harvesting real-time CPU, RAM, and Disk metrics across 100+ edge devices for centralized observability. - Resilience & Reliability: Implemented Circuit Breaker pattern (100 req/s limit) with Exponential Backoff and Jitter to absorb thundering-herd reconnect spikes, maintaining 99.9% uptime. - Performance Optimization: Reduced agent memory footprint by over 60% (under 15MB) using log rotation (10MB x 7) and memory-efficient data structures. - Security & Management: Secured endpoint telemetry with Bearer Token auth and SHA-256 HMAC integrity verification; built remote kill-switch mechanism.

Key Projects

EzSplit – Full-Stack AI Linguistic SaaS

Problem: Memorizing vocabulary without understanding etymological structure leads to rapid attrition.

Solution: Built full-stack SaaS with morpheme segmentation, SVG Bézier etymology trees, SM-2 Spaced Repetition flashcards, sub-350ms cached dictionary with async AI enrichment (Gemini 2.5 Flash Lite + DeepSeek fallback), and automated PayOS VietQR + PayPal checkout.

Result: Shipped to production with 51 static/dynamic routes, sub-second latency, and bilingual EN/VI localization.

[Live Demo: ezsplit.vercel.app](https://ezsplit.vercel.app)

High-Concurrency Inventory Engine

Problem: Peak flash-sale traffic causes concurrency race conditions and inventory overselling.

Solution: Single-Writer / Multi-Reader (SWMR) model with 100-connection reader pool, 1-connection writer queue guarded by Tx Mutex, and strict serializability (Stock = 1).

Result: Benchmarked at ~10,000 writes/sec on a single SQLite node with p95 latency < 0.1ms and zero overselling.

[GitHub Repository](#)

TapHoaNho – Retail Store Management ERP

Problem: Retail multi-branch operations require unified batch inventory tracking, supplier management, dynamic promotion policies, and POS checkout.

Solution: ASP.NET Core 9 Web API using Clean Architecture, Repository & Unit of Work pattern, AutoMapper, FluentValidation, and JWT + HttpOnly Cookies. Frontend built with React 19, TypeScript, TanStack Router (type-safe routing), TanStack Query, Zustand, and Ant Design.

Result: Strict domain isolation, end-to-end transactional consistency across orders/stock, and zero frontend runtime route mismatches.

[GitHub Repository](#)

Academy Hub – Gamified Telegram Mini App (TMA)

Problem: Web3 community onboarding requires native, low-friction learning experiences directly inside Telegram groups without browser redirection.

Solution: Telegram Mini App (TMA) featuring interactive bite-sized lessons, quizzes, XP & streak tracking, global leaderboard, and serverless Telegram Group role-based admin validation (api/check-admin.js).

Result: Zero-login in-chat app launch directly from Telegram bot commands with persistent user learning state.

[GitHub Repository](#)

Quant Trading & Due-Diligence Engine

Problem: Capture non-directional market yield and generate objective, un-hallucinated crypto asset evaluations.

Solution: Dual-engine trading system with Binance Cash & Carry delta-neutral funding rate arb (locking APR > 119% with kill-switch), HOSE ETF I-NAV arbitrage via DNSE API, and 7-factor hybrid due-diligence system combining CoinGecko quant scoring with multi-agent LLM analysis.

Result: 24/7 cloud execution with zero directional price risk, plus structured institutional research reports.

minPOS – Offline-First Micro-Merchant POS

Problem: Small retail/F&B merchants need an ultra-fast, affordable POS system that works reliably without internet.

Solution: Embedded Go backend with SQLite local storage, background transactional sync, thermal receipt printing, and hardware-bound license key generator.

Result: Sub-50ms transaction speeds, offline-first reliability, zero data loss.

[GitHub Repository](#)

Technical Skills

Languages: Go (Golang), TypeScript, JavaScript, C# (.NET), Rust, Java, C/C++, SQL

Frameworks & Backend: Go Fiber, ASP.NET Core 9, Next.js 15/16, React 19, Node.js, Express, Prisma ORM, Entity Framework Core, GORM, HTMX, Tailwind CSS v4, TanStack Router & Query

Databases & Caching: PostgreSQL, SQLite (WAL Mode), Redis, Neon

Architecture & Distributed Systems: High-Concurrency Architecture (SWMR), Circuit Breakers, WebSockets, Telegram Mini Apps (TMA), Repository & Unit of Work Pattern, Clean Architecture, SM-2 Spaced Repetition, Low-Latency Matching Engine

DevOps & Tools: Linux, Docker, GCP (Cloud Run, Compute Engine), Vercel, Git, CI/CD, OpenTelemetry, Puppeteer

Education

Saigon University (SGU) – Software Engineering (2021 - 2026)
Engineer Degree

Certifications

- TOEIC (760 R/L)
- AWS Cloud (In Progress)